Hosting Five Websites on One EC2 Instance with Nginx

Step-by-step deployment and architecture guide

Scope. Launch an Amazon EC2 instance, install Nginx, create five website document roots, bind each site to a different TCP port, and verify the result.

Created by **Mayur Chaudhari**

www.linkedin.com/in/mayur-chaudhari-devops

2026-10-11

Contents

1 1. Solution overview 1
1.1 What is deployed 1
1.2 Architecture diagram 1
2 2. Prerequisites and planning 1
2.1 Inbound security-group rules 2
3 3. Launch the EC2 instance 2
3.1 Connect over SSH 2
4 4. Install Nginx 2
5 5. Create the five website roots 3
5.1 Add simple test content 3
6 6. Configure Nginx for five sites 3
6.1 How the mapping works 4
7 7. Validate, restart, and verify 5
7.1 Linux command reference: what each command does 5
7.2 Browser tests 6
8 8. Troubleshooting guide 7
9 9. Operational and security considerations 7
10 10. Final checklist 8
11 Appendix A. Supplied command history 8

Reading note. The supplied file records the commands used on the server and the final Nginx server blocks. EC2 launch, security-group, and SSH steps below are the operational prerequisites needed to make that configuration reachable from the internet.

1 1. Solution overview

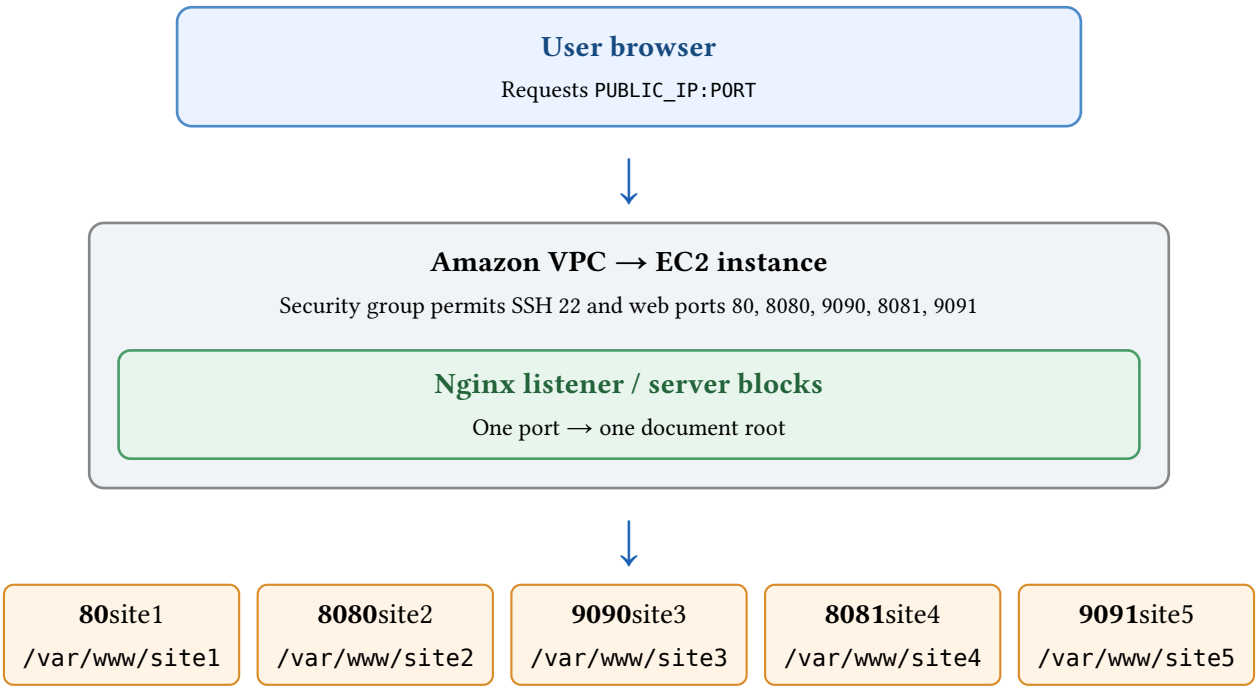
This setup hosts five independent static websites on one Amazon EC2 instance. Nginx is installed once and listens on five TCP ports. Each port maps to a separate directory containing an `index.html` file.

The supplied server hostname begins with `ip-172-31-0-185`, which is consistent with an EC2 private hostname. Use the instance's **public IPv4 address** or **public DNS name** when testing from your workstation; private addresses are reachable only inside the VPC or through an approved network path.

1.1 What is deployed

- **Compute:** one Amazon EC2 Linux instance.
- **Web server:** Nginx.
- **Content:** five static website roots under `/var/www`.
- **Routing model:** port-based routing, not domain-name-based routing.
- **Public endpoints:** ports 80, 8080, 9090, 8081, and 9091.

1.2 Architecture diagram



代码 1 Request flow from a browser to the five Nginx website roots.

2 2. Prerequisites and planning

Before running commands on the instance, collect the following values:

Item	Value / decision
AWS Region	The region where the EC2 instance will be launched.
Instance OS	Amazon Linux is implied by the supplied `yum` commands and `ec2-user` login.
Key pair	An existing EC2 key pair used for SSH access.

Public address	Public IPv4 or public DNS name used in browser tests.
Security group	Inbound rules for SSH and all five web ports.

2.1 Inbound security-group rules

Add only the access your environment needs. A practical lab configuration is:

Port	Protocol	Purpose / source recommendation
`22`	`TCP`	SSH administration; restrict to your IP where possible.
`80`	`TCP`	Website 1; open to `0.0.0.0/0` only if intended.
`8080`	`TCP`	Website 2.
`9090`	`TCP`	Website 3.
`8081`	`TCP`	Website 4.
`9091`	`TCP`	Website 5.

> **Important:** opening a port in Nginx does not automatically make it reachable. The EC2 security group, subnet network ACLs, and any host firewall must also allow the same traffic.

3 3. Launch the EC2 instance

. Open the AWS EC2 console and choose **Launch instance**. . Choose an Amazon Linux AMI. The supplied notes use `yum` and the `ec2-user` account, so select an Amazon Linux-compatible image. . Select an instance type appropriate for a small static-site lab, such as a burstable general-purpose instance. . Select or create a key pair. Download the private key once and store it securely. . In **Network settings**, select the VPC/subnet and attach a security group with the rules in Section 2. . Ensure the instance receives a public IPv4 address if it must be accessed directly from the internet. . Launch the instance and wait until its status checks pass. . Copy the public IPv4 address or public DNS name for the connection and browser tests.

3.1 Connect over SSH

From a terminal on your workstation, replace the placeholders and protect the key file permissions:

```
chmod 400 /path/to/key.pem
ssh -i /path/to/key.pem ec2-user@PUBLIC_DNS_OR_IP
```

If SSH fails, verify the instance is running, the address is current, the key pair matches, and inbound TCP port 22 is allowed from your source IP.

4 4. Install Nginx

The supplied command history starts with an operating-system update followed by Nginx installation:

```
sudo yum update -y
sudo yum install nginx -y
```

On an image where `yum` is unavailable, use the distribution's supported package manager instead (for example, `dnf`). The important result is that the `nginx` package is installed.
Enable and start Nginx so it comes up after a reboot:

```
sudo systemctl enable nginx
sudo systemctl start nginx
sudo systemctl status nginx
```

The original notes use `systemctl restart nginx` after configuration. `start` is used here for the first launch; later changes should be validated before restarting.

5 5. Create the five website roots

Create one directory per site and one entry page per directory. These commands are equivalent to the supplied history:

```
sudo mkdir -p /var/www/site1
sudo mkdir -p /var/www/site2
sudo mkdir -p /var/www/site3
sudo mkdir -p /var/www/site4
sudo mkdir -p /var/www/site5

sudo touch /var/www/site1/index.html
sudo touch /var/www/site2/index.html
sudo touch /var/www/site3/index.html
sudo touch /var/www/site4/index.html
sudo touch /var/www/site5/index.html
```

5.1 Add simple test content

The source notes indicate that each `index.html` was edited interactively with `vi`. You can use `vi`, another editor, or a heredoc. For a quick smoke test, write distinct content into each file:

```
for n in 1 2 3 4 5; do
  echo "<h1>Website $n</h1><p>Served by Nginx on EC2.</p>" \
    | sudo tee /var/www/site$n/index.html >/dev/null
done
```

For production content, replace these placeholders with complete HTML files and set ownership/permissions according to your deployment policy. Nginx needs read access to the document roots.

6 6. Configure Nginx for five sites

Create the configuration file requested in the supplied notes:

```
sudo vi /etc/nginx/conf.d/websites.conf
```

Paste the following server blocks. Each block listens on a unique port and serves a unique root:

```
server {
    listen 80;
    root /var/www/site1;
    index index.html;
}

server {
    listen 8080;
    root /var/www/site2;
    index index.html;
}

server {
    listen 9090;
    root /var/www/site3;
    index index.html;
}

server {
    listen 8081;
    root /var/www/site4;
    index index.html;
}

server {
    listen 9091;
    root /var/www/site5;
    index index.html;
}
```

Save the file and exit the editor. Because the file is under `/etc/nginx/conf.d/`, the main Nginx configuration normally includes it automatically.

6.1 How the mapping works

Port	Document root	Expected URL
80	/var/www/site1	http://PUBLIC_IP/
8080	/var/www/site2	http://PUBLIC_IP:8080/
9090	/var/www/site3	http://PUBLIC_IP:9090/
8081	/var/www/site4	http://PUBLIC_IP:8081/
9091	/var/www/site5	http://PUBLIC_IP:9091/

7 7. Validate, restart, and verify

The supplied notes correctly test the configuration before restarting Nginx:

```
sudo nginx -t
```

Expected result includes `syntax is ok` and `test is successful`. If the test fails, do not restart Nginx; fix the reported file and line first.
Inspect listening sockets before and after the restart:

```
sudo ss -lntp
sudo systemctl restart nginx
sudo ss -lntp
```

You should see Nginx listening on TCP ports 80, 8080, 8081, 9090, and 9091. The order in the `ss` output is not important.

7.1 Linux command reference: what each command does

The following reference explains the purpose of every Linux command used in this guide and in the supplied command history. The commands are grouped by the stage where they appear.

Command / syntax	Why it is used
`chmod 400 file`	Changes file permissions. `400` makes the SSH private key readable only by its owner, which SSH requires for security.
`ssh -i key user@host`	Opens a secure remote shell. `-i` selects the private key, while the final value identifies the remote user and EC2 address.
`sudo`	Runs the following command with administrator privileges. It is needed for system packages, service control, `/etc/nginx`, and `/var/www`.
`yum update -y`	Refreshes installed packages and applies available updates. `-y` automatically answers yes to the package manager prompts.
`yum install nginx -y`	Downloads and installs the Nginx web server package.
`systemctl enable nginx`	Registers Nginx to start automatically when the instance boots.
`systemctl start nginx`	Starts the Nginx service now, without waiting for a reboot.
`systemctl status nginx`	Shows whether the Nginx service is running and displays recent service information.
`systemctl restart nginx`	Stops and starts Nginx again so it reloads configuration changes. Validate with `nginx -t` first.
`mkdir -p path`	Creates a directory. `-p` also creates missing parent directories and avoids an error if the directory already exists.
`touch file`	Creates an empty file if it does not exist, or updates its timestamp if it already exists. Here it creates each `index.html` placeholder.

`vi file`	Opens the file in the `vi` text editor so HTML or Nginx configuration can be entered interactively.
`echo text`	Prints text to standard output. In the smoke-test loop it generates a small, distinct HTML page for each site.
`tee file`	Writes standard input to a file while optionally echoing it to the terminal. `sudo tee` allows a non-root shell to write into `/var/www`.
`|` (pipe)	Sends the output of the command on the left into the input of the command on the right; used to send `echo` output to `tee`.
`>/dev/null`	Redirects output to the null device, discarding it so the smoke-test loop stays quiet.
`for ...; do ...; done`	A shell loop. It repeats the enclosed commands for each site number or port in the list.
`nginx -t`	Checks Nginx configuration syntax and referenced files without restarting the service. It prevents a bad edit from being loaded.
`ss -lntp`	Lists listening TCP sockets and the processes using them. `-l` listening, `-n` numeric, `-t` TCP, and `-p` process details.
`ss -intp`	The original history uses this variant to inspect TCP sockets. `-i` adds internal TCP information; the other flags show numeric addresses and processes.
`curl -I URL`	Sends an HTTP request and prints response headers only. It is a quick local health check for each Nginx listener.

The configuration keywords in `websites.conf` are not shell commands: `server` defines a virtual server, `listen` selects its TCP port, `root` selects the directory to serve, and `index` names the default file.

7.2 Browser tests

Replace `PUBLIC_IP` with the instance's public address:

```
http://PUBLIC_IP/
http://PUBLIC_IP:8080/
http://PUBLIC_IP:9090/
http://PUBLIC_IP:8081/
http://PUBLIC_IP:9091/
```

Each URL should display a different page title or heading. You can also test from the instance itself with `curl`:

```
for port in 80 8080 9090 8081 9091; do
  echo "--- port $port ---"
  curl -I http://127.0.0.1:$port/
done
```

8 8. Troubleshooting guide

Symptom	Checks and remedy
Connection timeout	Confirm the security group permits the requested port and that the instance has a public route. Check host firewall rules if enabled.
502 / 503 response	This simple static setup should not need an upstream. Check the Nginx error log and configuration includes.
404 Not Found	Confirm the matching root contains `index.html`, paths are spelled correctly, and Nginx can read the files.
Only port 80 works	Open 8080, 8081, 9090, and 9091 in the security group; then confirm Nginx is listening with `sudo ss -lntp`.
`nginx -t` fails	Read the exact file and line in the error. Common causes are missing semicolons, braces, or an incorrect root path.
Permission denied	Inspect permissions on `/var/www` and the files. Grant read and directory-traverse access without making files unnecessarily writable.

9 9. Operational and security considerations

- **Prefer domain-based routing for production.** The supplied configuration separates sites by port, which is useful for learning and internal testing. For public sites, DNS names plus `server_name` blocks and usually ports 80/443 are easier for users.
- **Use HTTPS.** Put TLS certificates in place and redirect HTTP to HTTPS when these websites carry real user or business data.
- **Restrict SSH.** Allow port 22 from a trusted administrator IP or use AWS Systems Manager Session Manager where appropriate.
- **Limit exposure of nonstandard ports.** Ports 8080, 8081, 9090, and 9091 are public if the security group allows `0.0.0.0/0`. Narrow the source ranges for internal or test-only sites.
- **Back up configuration and content.** Preserve `/etc/nginx/conf.d/websites.conf` and the `/var/www/site*` directories in version control or an approved backup location.
- **Monitor the service.** Review Nginx access/error logs and use CloudWatch or another monitoring system for availability and resource usage.
- **Use least privilege.** Avoid broad write permissions on web roots. Deploy content through a controlled process.

10 10. Final checklist

- [ ] EC2 instance is running and status checks pass.
- [ ] SSH access works with the intended key pair.
- [ ] Security group allows SSH and the five required web ports.
- [ ] Nginx is installed, enabled, and running.
- [ ] `/var/www/site1` through `/var/www/site5` exist.
- [ ] Each site has its own `index.html`.
- [ ] `/etc/nginx/conf.d/websites.conf` contains the five server blocks.
- [ ] `sudo nginx -t` succeeds before restart.
- [ ] `sudo ss -lntp` shows all five Nginx listeners.
- [ ] All five browser URLs return the expected, distinct content.

11 Appendix A. Supplied command history

For traceability, the source file records this sequence on the server:

```
yum update -y
yum install nginx -y
mkdir -p /var/www/site1
mkdir -p /var/www/site2
mkdir -p /var/www/site3
mkdir -p /var/www/site4
mkdir -p /var/www/site5
touch /var/www/site1/index.html
touch /var/www/site2/index.html
touch /var/www/site3/index.html
touch /var/www/site4/index.html
touch /var/www/site5/index.html
vi /var/www/site5/index.html
vi /var/www/site1/index.html
vi /var/www/site2/index.html
vi /var/www/site3/index.html
vi /var/www/site4/index.html
sudo vi /etc/nginx/conf.d/websites.conf
sudo nginx -t
sudo ss -intp
sudo systemctl restart nginx
sudo ss -intp
```

The appendix preserves the original order; the main body adds the operational context, safer `sudo` usage for system paths, verification commands, and the EC2 network prerequisites needed for a reproducible deployment.